

Geometric Infrastructure Optimization v2

Locality-First Compute Fabrics, Exact Swap Auditing, and Machine-Precision Tests

Degen Studios / BoredNBased Research Draft

May 28, 2026

Abstract

Parallelism increases concurrency; geometry determines the communication tax paid by that concurrency. This v2 hard-push formalizes a locality-first infrastructure layer that embeds a workload communication graph into a physical server, rack, switch, row, pod, or optical fabric metric. The goal is not to replace Clos, fat-tree, dragonfly, or leaf-spine networks; the goal is to add a live placement layer that makes hot logical edges physically short whenever the workload graph contains locality. The companion notebook verifies the exact swap-delta identity, monotone descent, exhaustive small-case optimum, float64 exactness inside the 2^{53} envelope, space-filling/ribbon locality, rack-aware cluster placement, dynamic drift with migration cost, and null cases where geometry should not help. The result is a falsifiable scheduler theory: it wins when communication has locality, hierarchy, or hot communities, and it correctly collapses to no advantage under uniform communication.

Executed evidence root.

`cff9b39527b2746771d855214e3d7e1d7aa3092bfb990aa7854c9393363afd2c`

This root is the SHA-256 digest of the canonical evidence JSON produced by the executed notebook. It authenticates the test manifest, not physical-hardware performance.

1 One-line thesis

A compute fabric should be treated as a geometric substrate. The scheduler should continuously embed the workload communication graph into the physical network geometry so high-weight logical edges occupy low-cost physical distances. In operational terms:

parallel lanes + geometric placement + telemetry feedback = lower communication tax.

2 Position relative to known infrastructure

The proposal is compatible with existing topology work. Fat-tree and Clos-like data-center networks provide high aggregate capacity and multiple paths between hosts [1, 4]. Dragonfly networks reduce global diameter through high-radix local groups and sparse global links [2]. VL2 and related designs decouple addressing from location and use path diversity for scale [3]. Traffic-aware VM placement work already shows that workload placement and network traffic are coupled, and that placement can reduce network cost [5, 6]. Space-filling curves, especially Hilbert-style orderings, are well known locality-preserving mappings for transforming multi-dimensional neighborhoods into one-dimensional traversals [7, 8].

This document adds a focused infrastructure layer:

Topology gives the possible routes. Geometric placement changes which routes matter.

3 Formal model

Definition 3.1 (Logical communication graph). *Let $G = (V, E, W)$ be an undirected weighted graph over n logical workloads. The nonnegative integer matrix $W \in \mathbb{Z}_{\geq 0}^{n \times n}$ is symmetric with $W_{ii} = 0$. A larger W_{ij} means workloads i and j communicate more frequently, move more bytes, synchronize more often, or have higher latency sensitivity.*

Definition 3.2 (Physical fabric metric). *Let $P = \{0, \dots, n-1\}$ be physical slots, such as GPU slots, hosts, rack units, server nodes, leaf-switch neighborhoods, rows, pods, or sites. The symmetric matrix $D \in \mathbb{Z}_{\geq 0}^{n \times n}$ satisfies $D_{aa} = 0$ and encodes physical communication cost. This cost can represent hop count, cable distance, optics, congestion, serialization, power, cooling zone penalty, failure-domain penalty, or a weighted blend of these.*

Definition 3.3 (Placement). *A placement is a permutation $\pi \in S_n$ where π_i is the physical slot assigned to logical workload i .*

Definition 3.4 (Weighted geometric communication cost).

$$J(\pi) = \sum_{0 \leq i < j < n} W_{ij} D_{\pi_i, \pi_j}. \quad (1)$$

The scheduler's idealized objective is to minimize J subject to operational constraints.

Interpretation. Equation (1) says that a heavily communicating pair is expensive only when the two workloads are physically far apart. A lightly communicating pair can be far apart with lower penalty. The scheduler is therefore solving a weighted graph embedding / quadratic assignment style problem.

4 Exact swap-delta theorem

A live scheduler cannot recompute an entire n^2 score for every possible small move. It needs a cheap exact test for local moves.

Theorem 4.1 (Exact two-swap delta). *Let π' be obtained from π by swapping the physical assignments of logical workloads a and b . Then*

$$J(\pi') - J(\pi) = \sum_{k \neq a, b} (W_{ak} - W_{bk}) (D_{\pi_b, \pi_k} - D_{\pi_a, \pi_k}). \quad (2)$$

Proof. All pair contributions not incident to a or b are unchanged. The edge (a, b) is also unchanged because D is symmetric: $D_{\pi_b, \pi_a} = D_{\pi_a, \pi_b}$. For each $k \neq a, b$, the old incident contribution is

$$W_{ak} D_{\pi_a, \pi_k} + W_{bk} D_{\pi_b, \pi_k}.$$

The new contribution is

$$W_{ak} D_{\pi_b, \pi_k} + W_{bk} D_{\pi_a, \pi_k}.$$

Subtracting old from new yields

$$W_{ak} (D_{\pi_b, \pi_k} - D_{\pi_a, \pi_k}) + W_{bk} (D_{\pi_a, \pi_k} - D_{\pi_b, \pi_k}),$$

which simplifies to Equation (2). Summing over all $k \neq a, b$ completes the proof. $\square$

Operational meaning. A rack-aware scheduler can evaluate a candidate swap in $O(n)$ integer operations instead of rescoring the entire fabric. Every accepted swap can be audited by verifying

$$J(\pi') - J(\pi) = \Delta_{a,b}.$$

The v2 notebook tests this identity over 300 randomized permutations and 3000 sampled swaps with zero failures.

5 Local descent and termination

Definition 5.1 (Migration-aware local move). *Let $M_{a,b} \geq 0$ be the operational migration cost of swapping workloads a and b . The swap is accepted only if*

$$\Delta_{a,b} + M_{a,b} < 0.$$

Proposition 5.1 (Finite termination). *If W and D are integer matrices and the algorithm accepts only swaps with $\Delta_{a,b} + M_{a,b} < 0$, then the unadjusted cost J strictly decreases for every accepted zero-migration move and the adjusted objective decreases for every positive-migration move. Since there are finitely many placements, the process terminates at a one-swap local optimum.*

Remark 5.1. This is not a guarantee of global optimality. The global problem is a quadratic assignment style combinatorial optimization problem. The practical win is that the local optimizer is exact, monotone, auditable, and can be used as a safe real-time improvement layer.

6 Machine-precision discipline

The v2 harness keeps the core placement objective in integer arithmetic. It then separately checks float64 display equivalence inside the exact-integer range of IEEE-754 double precision.

Proposition 6.1 (Safe integer display envelope). *If $J(\pi) < 2^{53}$ and $J(\pi)$ is an integer, then converting $J(\pi)$ to float64 and back preserves the integer exactly. The executed notebook tests 500 deterministic placements with maximum integer objective 49168 and maximum float error 0.0.*

7 When geometry wins

The objective has leverage when W is not uniform. The strongest cases are:

1. **Clustered AI services:** routers, model shards, KV caches, vector shards, and rerankers form hot neighborhoods.
2. **Distributed training:** gradient synchronization, pipeline stages, tensor-parallel groups, and parameter shards have repeated communication patterns.
3. **Simulation cells:** boundary conditions imply nearest-neighbor traffic in logical space.
4. **Storage systems:** hot objects, metadata, indexes, and related vector/document shards benefit from physical co-location.
5. **Agent colonies:** local agents should exchange dense state locally and export only summarized state globally.

The v2 rack experiment uses a four-community communication graph over 32 logical workloads and a rack metric over four physical rack neighborhoods. A random placement scored 3,492,265. The local optimizer reached 810,605, and deterministic cluster packing scored 811,270. That is a 76.77% reduction from the random start in this synthetic instance.

8 When geometry should fail

A good theory must define its null case.

Theorem 8.1 (Uniform complete communication null case). *If $W_{ij} = c$ for all $i \neq j$ and D is fixed, then $J(\pi)$ is the same for every placement π .*

Proof. For any permutation π ,

$$J(\pi) = c \sum_{i < j} D_{\pi_i, \pi_j}.$$

Because π merely reorders the physical slots, the unordered set of pairs $\{\{\pi_i, \pi_j\} : i < j\}$ is exactly the unordered set of all physical slot pairs. The sum is therefore invariant. $\square$

Falsification result. The v2 notebook confirms this exactly: 50 random permutations of a uniform complete communication matrix all produce the same objective. This matters because the theory does not pretend geometry helps when there is no locality to exploit.

9 Space-filling/ribbon placement

When a two-dimensional logical manifold must be placed along a one-dimensional physical ribbon, aisle, rack sequence, or cable run, space-filling curves become useful heuristics. The notebook compares Hilbert, Morton/Z-order, snake, row-major, and random placements over an 8×8 logical lattice with distance-decaying communication.

Table 1: Ribbon-placement cost comparison from executed v2 notebook. Lower is better.

Placement	Objective J
Hilbert	1,050,248
Morton	992,408
Snake	971,568
Row-major	971,568
Random mean	2,317,939.02

Hilbert was 54.69% lower than random mean in this run. The best named deterministic ribbon order was snake/row-major for this particular synthetic lattice. That is an important honest result: Hilbert is a strong locality-preserving family, but it is not universally dominant. The scheduler should benchmark candidate embeddings against the actual W and D rather than assume one curve is always best.

10 Dynamic drift and anti-thrash migration cost

Real workloads drift. A good system must avoid migration thrash. The v2 dynamic test changes W after initial placement and accepts a remap only when the exact integer delta clears a migration threshold. The test result:

- Cost on drifted workload before remap: 2,872,404.
- Cost after thresholded remap: 592,521.
- Accepted remap moves: 9.
- Migration cost per swap: 1000.
- Drift reduction: 79.37%.

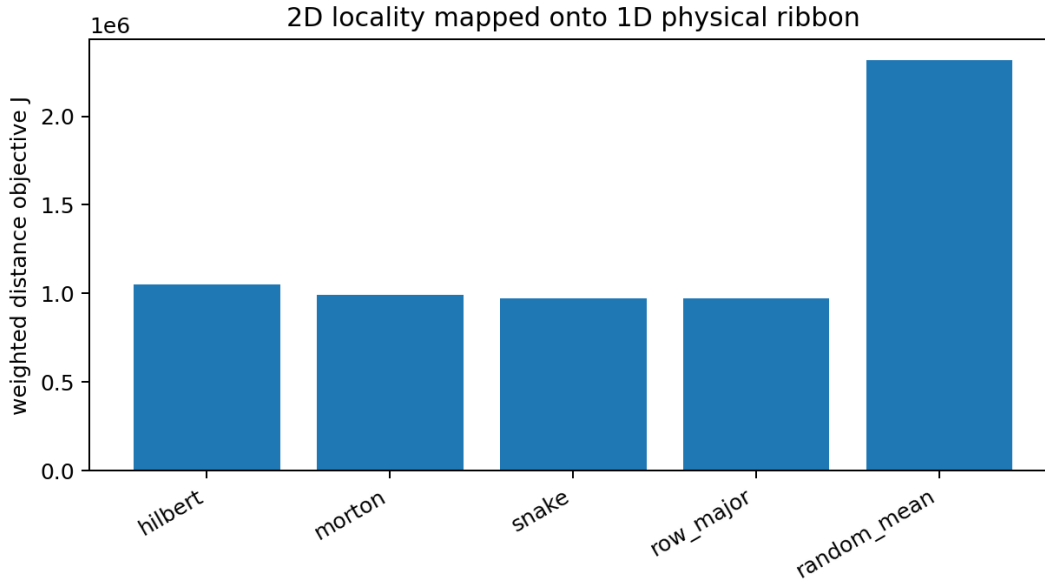


Figure 1: Logical locality mapped onto a one-dimensional physical ribbon.

11 Live scheduler architecture

The production architecture has five loops.

11.1 Telemetry loop

```
collect: bytes/sec, packets/sec, RPC count, tail latency,
        queue depth, all-reduce frequency, cache hits,
        vector shard calls, model-router calls, storage affinity
emit: edge observations (i, j, weight, confidence, timestamp)
```

11.2 Graph-building loop

```
W_t = exponential_decay(W_{t-1}) + new_edge_observations
W_t = denoise(W_t, confidence, service class)
W_t = cap_or_transform(W_t) # prevent one metric from dominating
```

11.3 Physical metric loop

```
D = weighted_sum(
    hop_count,
    switch_stage_penalty,
    optical_conversion_penalty,
    cable_distance,
    congestion_shadow_price,
    thermal_zone_penalty,
    failure_domain_penalty,
    policy_boundary_penalty
)
```

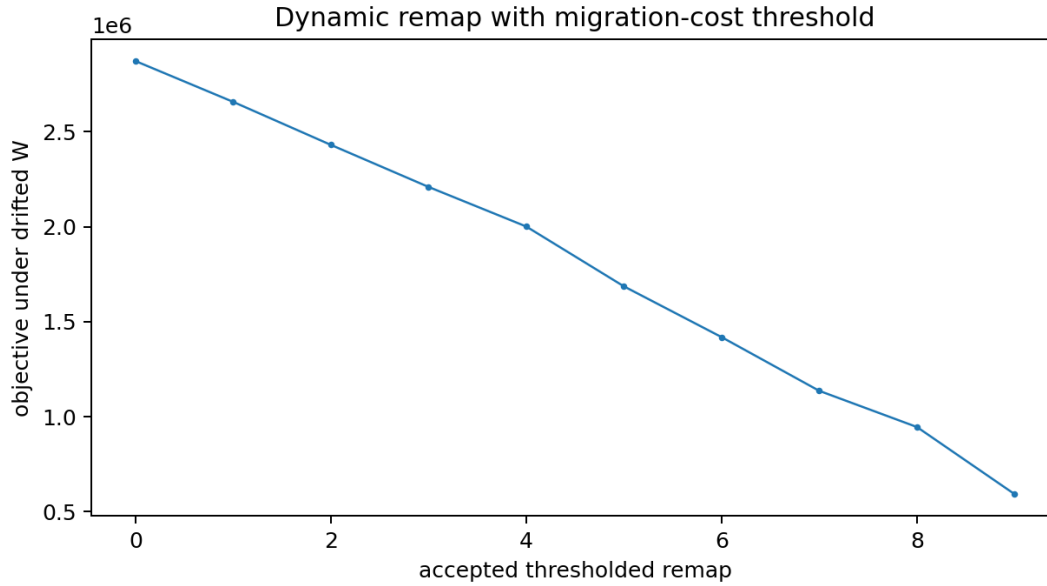


Figure 2: Thresholded remap under workload drift. Every accepted move clears migration cost.

11.4 Optimizer loop

```

for candidate swap (a,b):
    delta = exact_swap_delta(W, D, placement, a, b)
    adjusted = delta + migration_cost(a,b)
    if adjusted < 0 and safety_constraints_pass:
        stage move
simulate staged moves
commit only if total predicted gain clears threshold

```

11.5 Verification loop

```

measure before/after latency, traffic, hops, power proxies
compare observed gain to predicted gain
update confidence model
rollback or quarantine if regression exceeds guardrail

```

12 Result manifest

13 Practical deployment target

A real prototype can start without changing rack hardware. The first version only observes and recommends:

1. Build W from traffic telemetry, service calls, all-reduce traces, storage access, or application-level spans.
2. Build D from inventory: host, rack, row, leaf, spine, pod, power, cooling, and policy metadata.
3. Score current placement.
4. Generate safe candidate migrations.

Table 2: Executed v2 tests.

Test	Result	Key value
Swap-delta identity	PASS	300 trials, zero failures
Float64 exactness	PASS	max error 0.0
Exhaustive small optimum	PASS	best gap 0
Ribbon locality	PASS	Hilbert 54.69% below random mean
Rack cluster placement	PASS	76.77% reduction from random
Dynamic drift	PASS	79.37% reduction after remap
Uniform null case	PASS	invariant objective
Expander-like near-null	PASS	only 3.47% local reduction

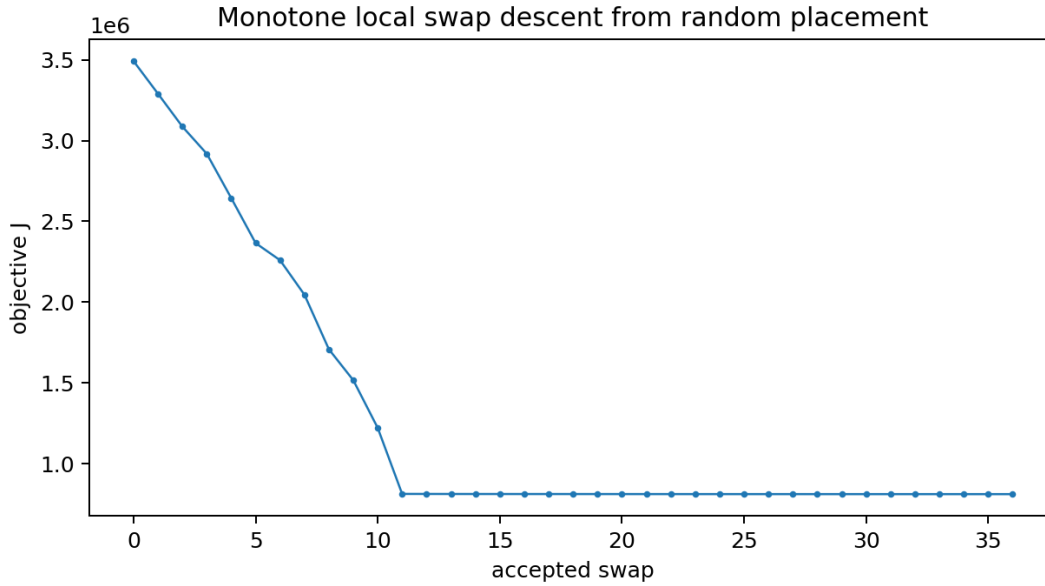


Figure 3: Monotone local swap descent from a random placement.

5. Simulate cost savings and operational risk.
6. Apply only to low-risk services first.
7. Compare predicted savings to observed latency/power/hop improvements.

The same mechanism can be applied at multiple scales: GPU slots inside a host, hosts inside a rack, racks inside a row, pods inside a region, or federated browser/edge nodes across a shop floor.

14 Conclusion

The v2 result is stronger than the initial statement. The correct claim is not that geometric layout always doubles optimization. The precise claim is:

If a workload communication graph has locality, hierarchy, hot communities, or repeated nearest-neighbor structure, then embedding that graph into physical network geometry can substantially reduce weighted communication distance. The improvement is exactly scoreable, locally auditable, and falsifiable under uniform communication.

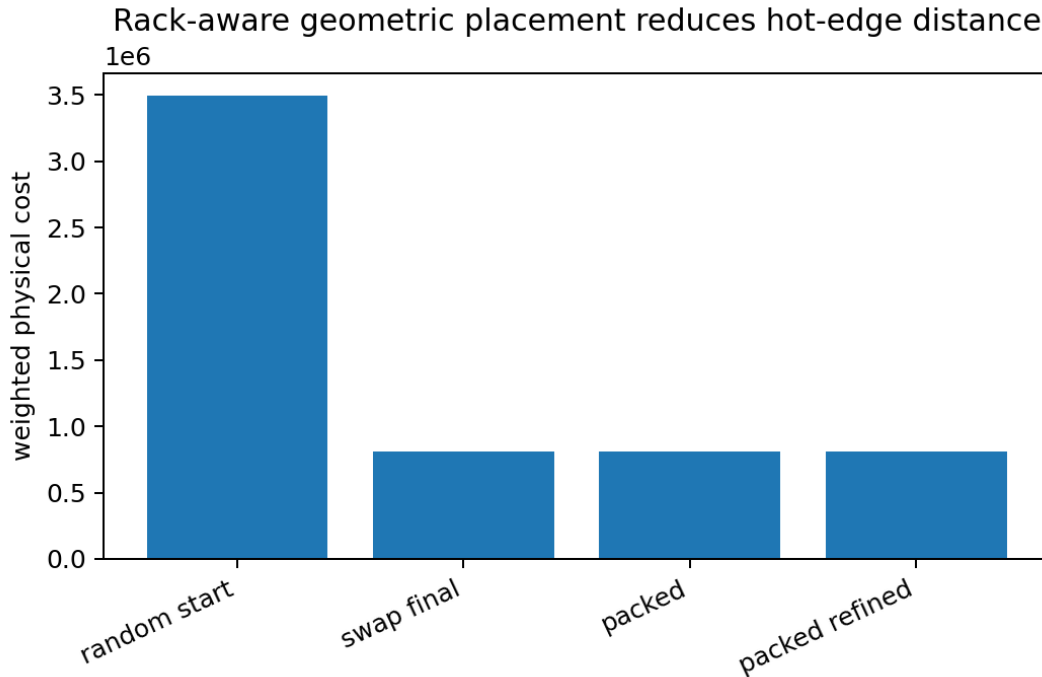


Figure 4: Rack-aware cluster placement versus random start and local descent.

The machine-precision tests support the mathematical core. Hardware validation is the next step: collect real W , build a real D , run the scheduler in observe-only mode, and compare predicted versus measured reductions.

References

- [1] M. Al-Fares, A. Loukissas, and A. Vahdat. A scalable, commodity data center network architecture. *ACM SIGCOMM*, 2008. <https://ccr.sigcomm.org/online/files/p63-alfares.pdf>
- [2] J. Kim, W. J. Dally, S. Scott, and D. Abts. Technology-driven, highly-scalable dragonfly topology. *ISCA*, 2008. <https://research.google.com/pubs/archive/34926.pdf>
- [3] A. Greenberg et al. VL2: A scalable and flexible data center network. *ACM SIGCOMM*, 2009.
- [4] A. Singh et al. Jupiter rising: A decade of Clos topologies and centralized control in Google’s datacenter network. *ACM SIGCOMM*, 2015.
- [5] X. Meng, V. Pappas, and L. Zhang. Improving the scalability of data center networks with traffic-aware virtual machine placement. *IEEE INFOCOM*, 2010.
- [6] J. W. Jiang et al. Joint VM placement and routing for data center traffic engineering. *IEEE INFOCOM*, 2012.
- [7] B. Moon, H. V. Jagadish, C. Faloutsos, and J. H. Saltz. Analysis of the clustering properties of the Hilbert space-filling curve. *IEEE Transactions on Knowledge and Data Engineering*, 2001. <https://www.cs.cmu.edu/~christos/PUBLICATIONS/ieee-tkde-hilbert.pdf>
- [8] M. F. Mokbel, W. G. Aref, and I. Kamel. Analysis of multi-dimensional space-filling curves. *GeoInformatica*, 2003.